Nama: Hans Malvin Djojo

https://github.com/hansmalvin/LastMile

```
Environment Setup
 Create a Project Directory: Create a new directory for your MERN application.
Name it something descriptive like mern-task-manager.



bash
mkdir mern-task-manager
cd mern-task-manager

 Initialize the Backend (Node.js/Express.js): Create a backend directory and
initialize a Node.js project.



bash
mkdir backend
cd backend
npm init -y

    This command creates a package.json file with default settings.
 Install Backend Dependencies: Install necessary packages for the backend:
Express.js, Mongoose (for MongoDB interaction), and CORS (for handling Cross-
Origin Resource Sharing).



bash
npm install express mongoose cors

 Create the Frontend (React): Navigate back to the root directory and create a
new React application using create-react-app.



bash
cd ..
npx create-react-app frontend

    This command sets up a basic React project structure.
 Install Frontend Dependencies: Install axios for making HTTP requests to the
backend API.  Navigate into the frontend directory first.
```

```
bash
cd frontend
npm install axios

 Project Structure:  Your project structure should look like this:



mern-task-manager/
├── backend/
│ ├── node_modules/
│ ├── package.json
│ ├── server.js // Main server file (to be created)
│ └── models/ // Directory for Mongoose models (to be created)
│ └── task.js // Task model (to be created)
├── frontend/
│ ├── node_modules/
│ ├── package.json
│ ├── public/
│ ├── src/
│ │ ├── App.js // Main App component
│ │ ├── components/ // Directory for React components (to be created)
│ │ │ └── TaskList.js // React component to display task list (to be created)
│ │ └── App.css
│ └── README.md
└── README.md

Step-by-Step Instructions
Step 1: Set up the Backend Server (Node.js/Express.js)
 Create server.js: In the backend directory, create a file named server.js.


 Implement Basic Server: Add the following code to server.js to set up a basic
Express.js server.



javascript
// backend/server.js
const express = require('express');
const cors = require('cors');
const mongoose = require('mongoose');
const app = express();
const port = process.env.PORT || 5000;

app.use(cors());
app.use(express.json()); // Middleware to parse JSON bodies
```

```
const uri = 'mongodb://127.0.0.1:27017/mern-task-manager'; // Replace with
your MongoDB URI
mongoose.connect(uri, { useNewUrlParser: true, useUnifiedTopology: true });

const connection = mongoose.connection;
connection.once('open', () => {
console.log("MongoDB database connection established successfully");
});

// Define a simple route for testing
app.get('/', (req, res) => {
res.send('MERN Task Manager Backend is running!');
});

app.listen(port, () => {
console.log(Server is running on port: ${port});
});


    Explanation: This code initializes an Express.js server, configures CORS,
sets up middleware to parse JSON request bodies, connects to a MongoDB
database using Mongoose, and defines a basic route. Make sure your MongoDB
server is running.  Replace mongodb://127.0.0.1:27017/mern-task-manager with
your MongoDB connection string if using MongoDB Atlas or a different local
configuration.
 Run the Server: In the backend directory, run the server.



bash
node server.js

    Expected Output:


MongoDB database connection established successfully
Server is running on port: 5000

    This indicates that the server is running and connected to the MongoDB
database.  If you see errors, double-check your MongoDB connection string and
ensure MongoDB is running.
Step 2: Define the Task Model (Mongoose)
 Create task.js: In the backend/models directory, create a file named task.js.


 Define the Task Schema: Add the following code to task.js to define the
Mongoose schema for a task.
```

```
javascript
// backend/models/task.js
const mongoose = require('mongoose');
const Schema = mongoose.Schema;

const taskSchema = new Schema({
description: {
type: String,
required: true,
trim: true,
minlength: 3
},
completed: {
type: Boolean,
default: false
}
}, {
timestamps: true, // Automatically adds createdAt and updatedAt fields
});

const Task = mongoose.model('Task', taskSchema);

module.exports = Task;
```

Explanation: This code defines a Mongoose schema for a Task with fields for description (a string that is required, trimmed, and has a minimum length of 3) and completed (a boolean with a default value of false). The timestamps option automatically adds createdAt and updatedAt fields to the documents.

Step 3: Implement the Task API (CRUD Operations)

Create routes directory (Optional but recommended): In the backend directory, you can create a directory named routes and add tasks.js to keep your server.js cleaner. If you skip this, put the code below directly into server.js.

Create tasks.js (or modify server.js): Add the following code to define the API endpoints for creating, reading, updating, and deleting tasks.

```
javascript
// backend/routes/tasks.js (or in server.js)
const router = require('express').Router();
let Task = require('../models/task');
// Get all tasks
router.route('/').get((req, res) => {
Task.find()
```

```
.then(tasks => res.json(tasks))
.catch(err => res.status(400).json('Error: ' + err));
});

// Add a new task
router.route('/add').post((req, res) => {
const description = req.body.description;
const newTask = new Task({description});

newTask.save()
.then(() => res.json('Task added!'))
.catch(err => res.status(400).json('Error: ' + err));
});

// Get a task by ID
router.route('/:id').get((req, res) => {
Task.findById(req.params.id)
.then(task => res.json(task))
.catch(err => res.status(400).json('Error: ' + err));
});

// Delete a task by ID
router.route('/:id').delete((req, res) => {
Task.findByIdAndDelete(req.params.id)
.then(() => res.json('Task deleted.'))
.catch(err => res.status(400).json('Error: ' + err));
});

// Update a task by ID
router.route('/update/:id').post((req, res) => {
Task.findById(req.params.id)
.then(task => {
task.description = req.body.description;
task.completed = req.body.completed;

task.save()
.then(() => res.json('Task updated!'))
.catch(err => res.status(400).json('Error: ' + err));
})
.catch(err => res.status(400).json('Error: ' + err));
});

module.exports = router;


    Explanation: This code defines API endpoints for retrieving all tasks (GET
/), adding a new task (POST /add), retrieving a task by ID (GET /:id),
deleting a task by ID (DELETE /:id), and updating a task by ID (POST
/update/:id).  It uses Mongoose to interact with the MongoDB database.
```

Mount the Routes in server.js: If you created backend/routes/tasks.js, you need to import and use these routes in your main server.js file:

javascript

```
// backend/server.js
const express = require('express');
const cors = require('cors');
const mongoose = require('mongoose');
const app = express();
const port = process.env.PORT || 5000;

app.use(cors());
app.use(express.json()); // Middleware to parse JSON bodies

const uri = 'mongodb://127.0.0.1:27017/mern-task-manager'; // Replace with your MongoDB URI
mongoose.connect(uri, { useNewUrlParser: true, useUnifiedTopology: true });

const connection = mongoose.connection;
connection.once('open', () => {
console.log("MongoDB database connection established successfully");
});

const tasksRouter = require('./routes/tasks'); // Import the tasks route
app.use('/tasks', tasksRouter); // Mount the tasks route at /tasks

app.listen(port, () => {
console.log(Server is running on port: ${port});
});
```

Explanation: This imports the tasks router and mounts it at the /tasks path. This means that all routes defined in tasks.js will be prefixed with /tasks.

Test the API Endpoints: Use Postman or Insomnia to test the API endpoints.

- POST http://localhost:5000/tasks/add with a JSON body like {"description": "Buy groceries"} to create a new task.
- GET http://localhost:5000/tasks/ to retrieve all tasks.
- GET http://localhost:5000/tasks/:id (replace :id with a valid task ID) to retrieve a specific task.
- POST http://localhost:5000/tasks/update/:id (replace :id with a valid task ID) with a JSON body like {"description": "Buy groceries and milk", "completed": true} to update a task.
- DELETE http://localhost:5000/tasks/:id (replace :id with a valid task ID) to delete a task.

Step 4: Develop the React Frontend

Modify App.js: In the frontend/src directory, open App.js and replace its content with the following code.

javascript

```
// frontend/src/App.js
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import './App.css';
function App() {
const [tasks, setTasks] = useState([]);
const [newTask, setNewTask] = useState('');

useEffect(() => {
fetchTasks();
}, []);

const fetchTasks = async () => {
try {
const response = await axios.get('http://localhost:5000/tasks');
setTasks(response.data);
} catch (error) {
console.error('Error fetching tasks:', error);
}
};

const addTask = async () => {
try {
await axios.post('http://localhost:5000/tasks/add', { description: newTask });
setNewTask('');
fetchTasks();
} catch (error) {
console.error('Error adding task:', error);
}
};

const deleteTask = async (id) => {
try {
await axios.delete(http://localhost:5000/tasks/${id});
fetchTasks();
} catch (error) {
console.error('Error deleting task:', error);
}
};

return (

MERN Task Manager
```

```
type="text"
value={newTask}
onChange={(e) => setNewTask(e.target.value)}
placeholder="Add a new task"
/>
Add Task
{tasks.map(task => (
{task.description}
deleteTask(task._id)}>Delete
))}
);
}
export default App;
```

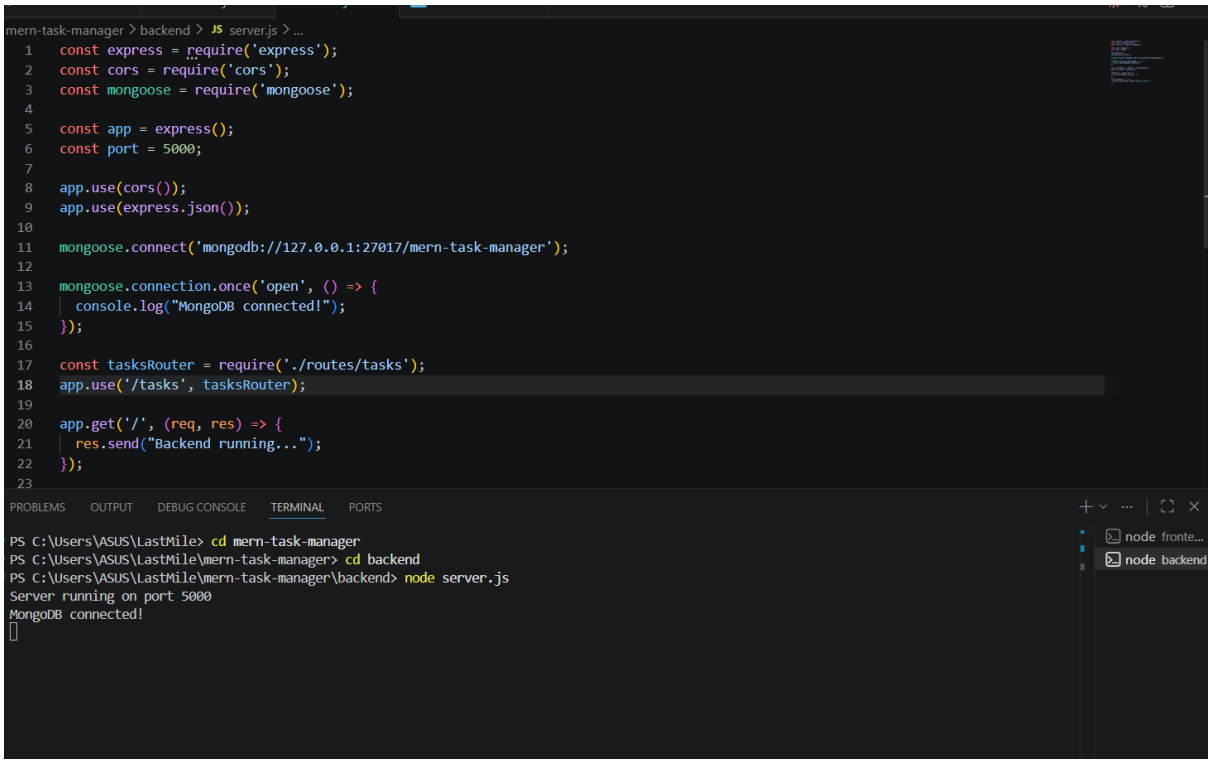
Explanation: This code defines a React component that fetches tasks from the backend API, displays them in a list, and allows users to add and delete tasks. It uses axios to make HTTP requests.

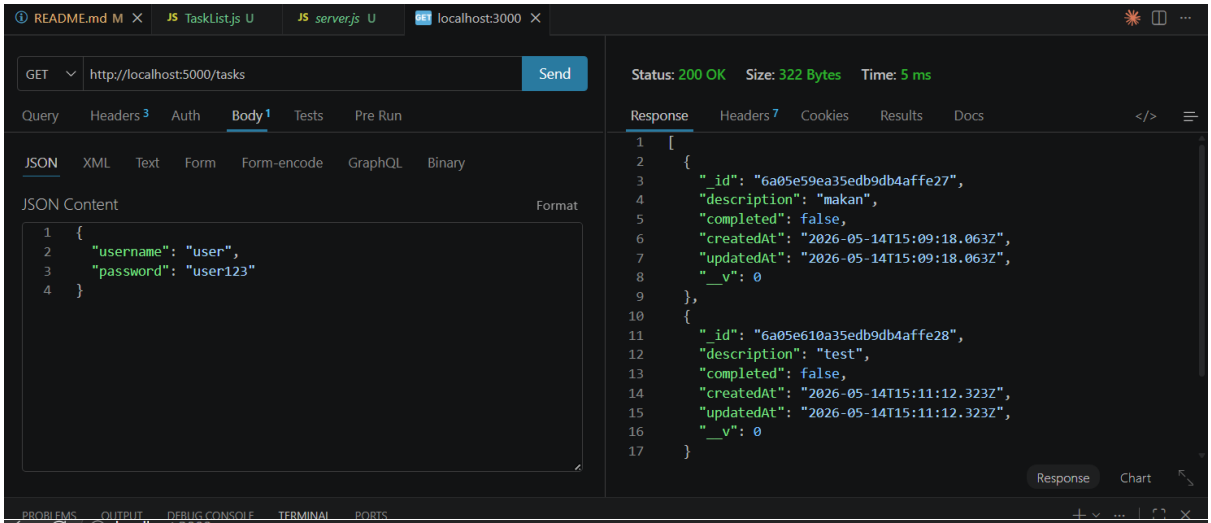
Add Basic Styling (Optional): In frontend/src/App.css, add some basic styling to improve the appearance of the application.

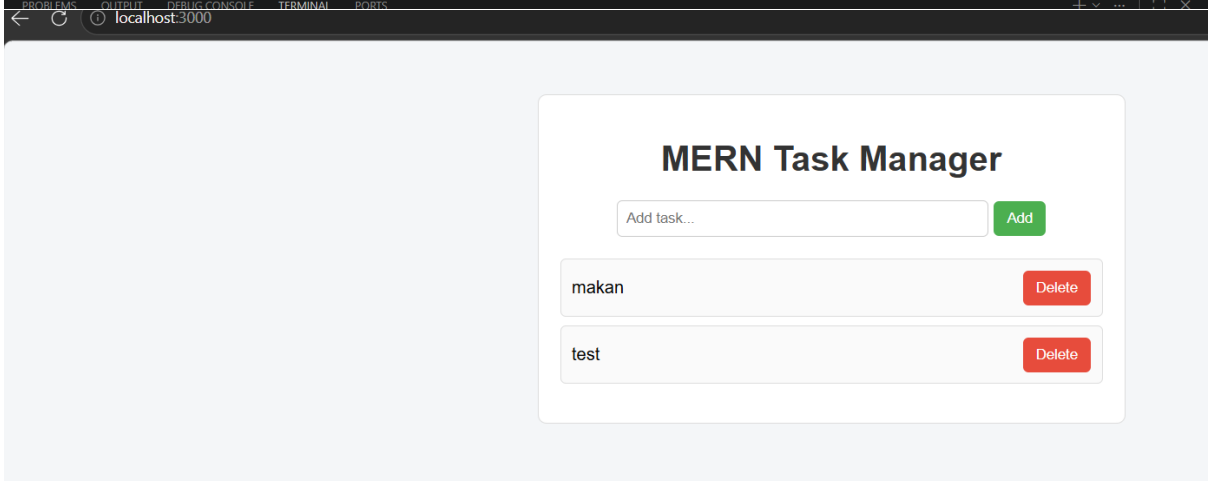
```
css
/ frontend/src/App.css /
.App {
font-family: sans-serif;
text-align: center;
}
input[type="text"] {
padding: 8px;
margin-right: 10px;
border: 1px solid #ccc;
border-radius: 4px;
}

button {
padding: 8px 12px;
background-color: #4CAF50;
color: white;
border: none;
border-radius: 4px;
cursor: pointer;
}

ul {
list-style: none;
padding: 0;
}
```

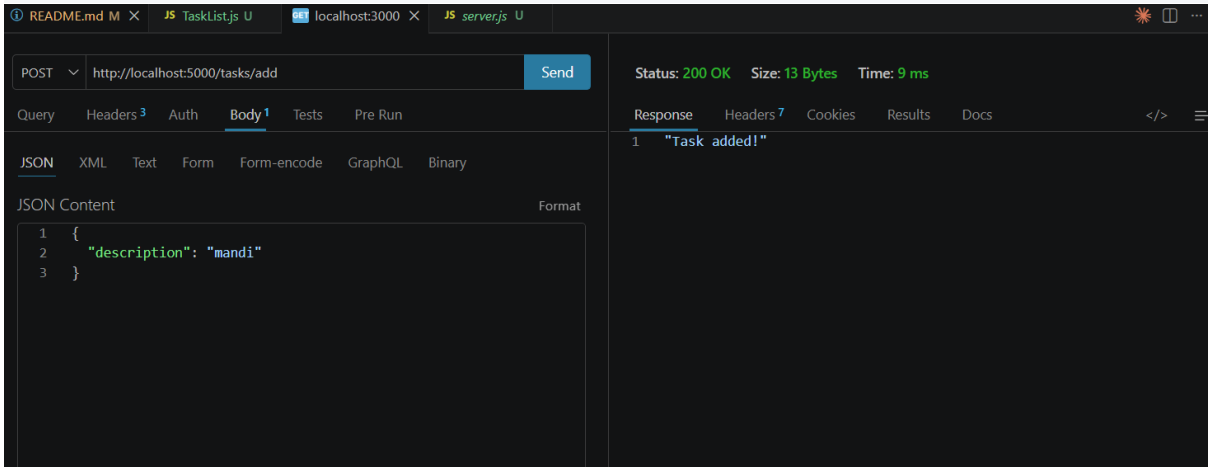
```
li {
padding: 8px;
border-bottom: 1px solid #eee;
}


 Run the Frontend: In the frontend directory, run the React application.



bash
npm start

    Expected Output:  A browser window will open displaying the React
applicatio
```

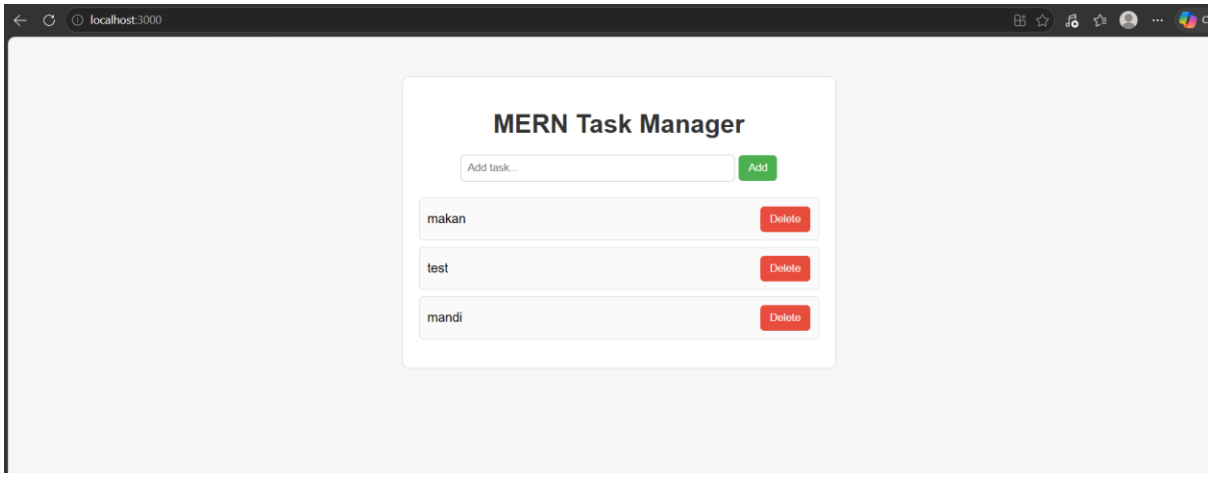
Answer:



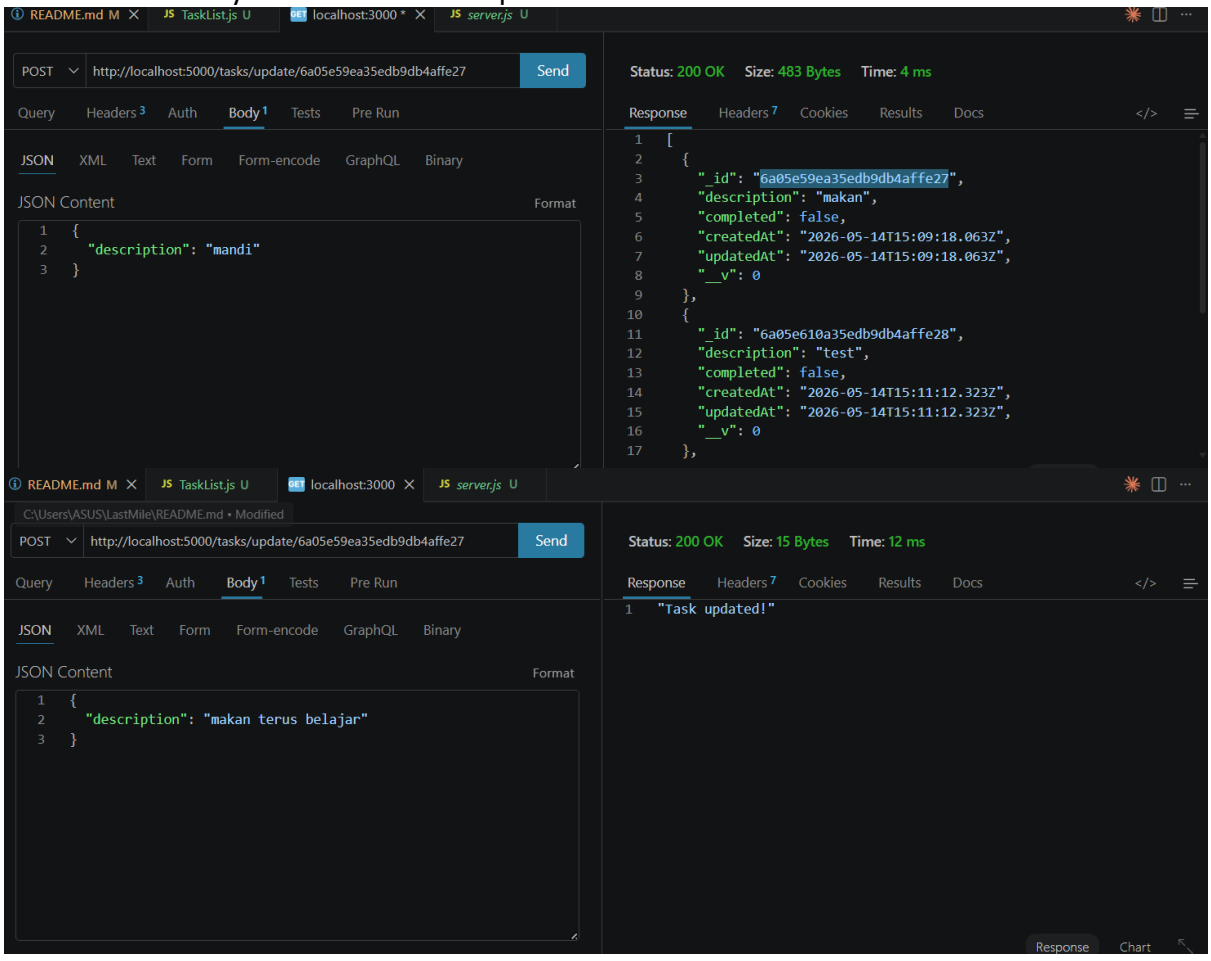

README.md M
TaskList.js U
server.js U
localhost:3000
GET
http://localhost:5000/tasks
Send
Query
Headers 3
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
{
"username": "user",
"password": "user123"
}
Status: 200 OK
Size: 322 Bytes
Time: 5 ms
Response
Headers 7
Cookies
Results
Docs
[
{
"_id": "6a05e59ea35edb9db4affe27",
"description": "makan",
"completed": false,
"createdAt": "2026-05-14T15:09:18.063Z",
"updatedAt": "2026-05-14T15:09:18.063Z",
"__v": 0
},
{
"_id": "6a05e610a35edb9db4affe28",
"description": "test",
"completed": false,
"createdAt": "2026-05-14T15:11:12.323Z",
"updatedAt": "2026-05-14T15:11:12.323Z",
"__v": 0
}
Response
Chart
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS


localhost:3000
MERN Task Manager
Add task...
Add
makan
Delete
test
Delete

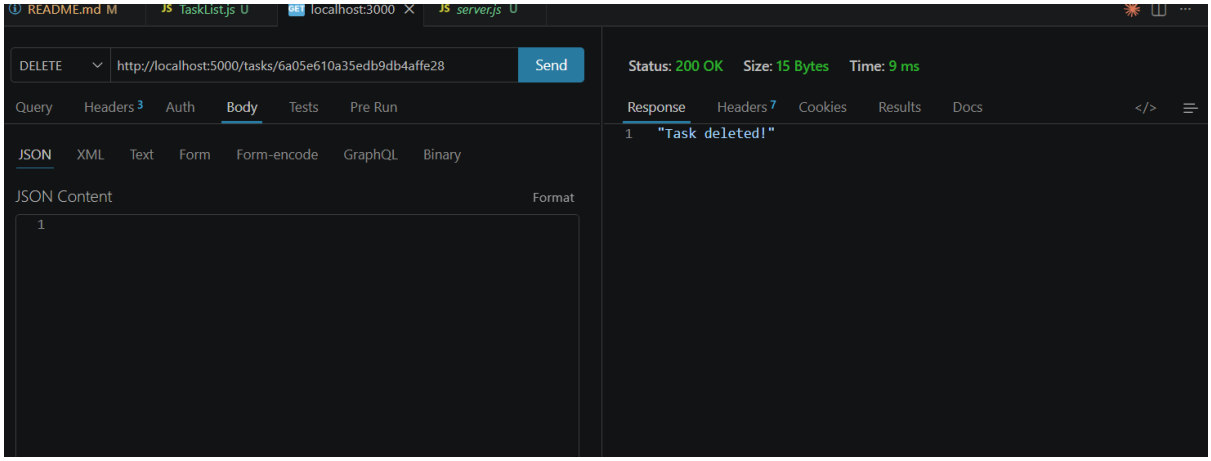

README.md M
TaskList.js U
localhost:3000
server.js U
POST
http://localhost:5000/tasks/add
Send
Query
Headers 3
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
{
"description": "mandi"
}
Status: 200 OK
Size: 13 Bytes
Time: 9 ms
Response
Headers 7
Cookies
Results
Docs
"Task added!"



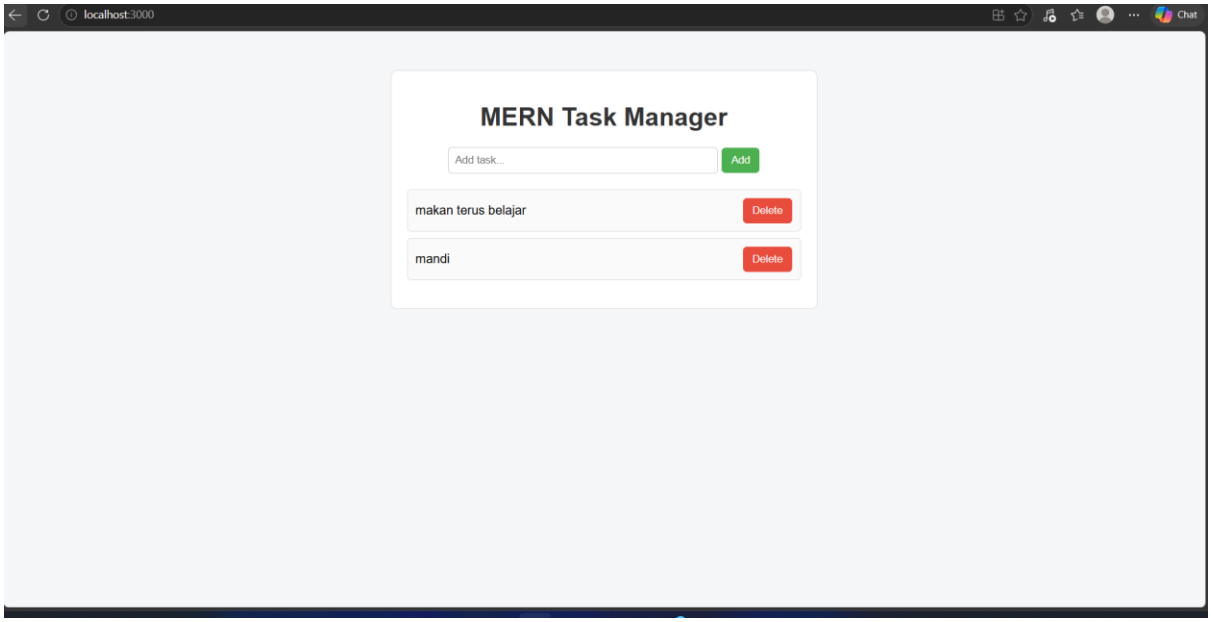
Ambil makan id nya untuk dilakukan update



Delete test dengan id nya


README.md M
TaskList.js U
localhost:3000
server.js U
DELETE
http://localhost:5000/tasks/6a05e610a35edb9db4affe28
Send
Status: 200 OK
Size: 15 Bytes
Time: 9 ms
Query
Headers 3
Auth
Body
Tests
Pre Run
Response
Headers 7
Cookies
Results
Docs
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
1
"Task deleted!"

Hasil akhir


localhost:3000
Chat
MERN Task Manager
Add task...
Add
makan terus belajar
Delete
mandi
Delete